

Hausarbeit

Okan Ayhan (30433120)

Fachhochschule Südwestfalen

Schriftliche Ausarbeitung in dem Modul

„Datenanalyse im Unternehmen“

27. September 2026

Inhaltsverzeichnis

1	Einleitung und Problemstellung	1
2	Theorieteil	2
2.1	KI/LLM-Technologien für Prozessautomatisierung	2
2.2	Architekturmuster: Orchestrierung vs. Choreografie	3
2.3	Human-in-the-loop und Automatisierungsgrade	4
3	Beschreibung des Prozesses	4
3.1	Typischer Ablauf einer Fallbearbeitung in Versicherungsagenturen	4
3.2	Automatisierbare vs. nicht-automatisierbare Prozessschritte	6
3.3	Veranschaulichung anhand eines Beispielszenarios	6
4	Implementierung und Architektur	8
4.1	Entwicklungsvorgehen und Eigenanteil	8
4.2	Systemarchitektur: Services und Ordnerstruktur	9
4.3	Choreografie und LLM-Einsatz am Beispielszenario	10
4.4	Endzustände als Umsetzung von Human-in-the-loop	11
5	Grenzen und Ausblick	11
5.1	Grenzen	12
5.2	Ausblick	12
6	Fazit	13
	Quellenverzeichnis	14

1 Einleitung und Problemstellung

Der deutsche Versicherungsmarkt steht unter erheblichem Digitalisierungsdruck. Kundinnen und Kunden erwarten heute schnelle Reaktionszeiten auf Schadenmeldungen oder Serviceanfragen, während viele Versicherer und ihre Vertriebsorganisationen diese Prozesse noch weitgehend manuell abwickeln. Besonders deutlich wird das in Exklusivagenturen, die als kleinste operative Einheit direkt mit den Kundinnen und Kunden in Kontakt stehen und dabei eine hohe Zahl an wiederkehrenden, strukturell ähnlichen Vorgängen bearbeiten müssen. Genau an dieser Stelle setzt die vorliegende Arbeit an: Sie untersucht, wie sich Künstliche Intelligenz (KI), insbesondere Large Language Models (LLMs), einsetzen lässt, um Geschäftsprozesse in solchen Agenturen zu unterstützen und teilweise zu automatisieren.

Die Themenwahl hatte einen konkreten Auslöser. Sie entstand im Rahmen eines Bewerbungsprozesses bei einem Versicherungsunternehmen. Im Mittelpunkt stand die Frage, wie sich KI in dessen Exklusivagenturen gewinnbringend einsetzen lässt, um bestehende Prozesse zu verbessern oder ganz zu automatisieren. Diese Fragestellung bildet die eigentliche Problemstellung der Arbeit: nicht ob KI in diesem Umfeld grundsätzlich anwendbar ist, sondern an welchen Stellen des Agenturalltags sie tatsächlich einen Unterschied macht und wo ihr Einsatz an Grenzen stößt.

Um diese Frage seriös beantworten zu können, reicht technisches Wissen über LLMs allein nicht aus. Wie eine Exklusivagentur tatsächlich arbeitet, welche Anfragen dort täglich eingehen und wie Sachbearbeiterinnen und Sachbearbeiter mit ihnen umgehen, lässt sich als Außenstehender kaum korrekt einschätzen. Um einen Einblick in diese internen Prozesse zu erhalten, fand ein Online-Meeting mit einer Person statt, die selbst in einer solchen Agentur tätig ist. Dieser Kontakt wurde von dem Unternehmen, bei dem das Bewerbungsgespräch stattfand, extra für die Informationssammlung im Rahmen dieser Arbeit organisiert. Dieser Austausch ersetzt keine systematische empirische Erhebung, lieferte aber die praktische Grundlage, auf der die in dieser Arbeit getroffenen Annahmen über typische Arbeitsabläufe beruhen.

Eine Exklusivagentur vertritt im Unterschied zum unabhängigen Makler ausschließlich einen einzigen Versicherer und tritt gegenüber der Kundschaft in dessen Namen auf. Organisatorisch handelt es sich meist um kleine, inhabergeführte Betriebe. Es gibt eine Agenturleitung, ergänzt um wenige Sachbearbeiterinnen und Sachbearbeiter, manchmal auch nur eine Einzelperson mit Unterstützung durch Aushilfen. Das Tagesgeschäft gliedert sich grob in drei Bereiche. Es gibt das Neugeschäft und die Vertragsberatung, laufende Bestandspflege sowie die Bearbeitung eingehender Kundenanliegen. Darunter fallen sowohl Schadenmeldungen als auch allgemeine Serviceanfragen wie Adressänderungen oder Auskunftswünsche. Gerade Letztere machen einen großen Teil des Tagesgeschäfts aus und folgen dabei erkennbar wiederkehrenden Mustern, was sie zu einem naheliegenden Ausgangspunkt für Automatisierung macht.

Angesichts der Breite möglicher Anwendungsfälle war eine Eingrenzung notwendig. Die Arbeit konzentriert sich auf zwei Bereiche: die Bearbeitung von Schadenmeldungen und die Bearbeitung von Serviceanfragen, im weiteren Verlauf als Schaden- und Support-Prozess bezeichnet. Beide Bereiche sind hochfrequent, folgen einem weitgehend standardisierbaren Ablauf und eignen sich damit gut, um den Einsatz von LLMs an konkreten Entscheidungs-

punkten zu untersuchen, statt KI als diffuses Gesamtkonzept zu behandeln. Auf dieser Eingrenzung baut auch der im Rahmen dieser Arbeit entwickelte Software-Prototyp auf, der in den späteren Kapiteln als Implementierungsbeleg dient.

Der weitere Aufbau der Arbeit folgt dieser Logik vom Allgemeinen zum Konkreten. Kapitel 2 legt die theoretischen Grundlagen. Es ordnet den Einsatz von LLMs zur Prozessautomatisierung ein, grenzt Orchestrierung von Choreografie als Architekturprinzipien ab und diskutiert das Konzept des Human-in-the-loop bei automatisierten Entscheidungen. Kapitel 3 beschreibt den typischen Ablauf von Schaden- und Serviceprozessen in einer Exklusivagentur, wie er sich aus dem geführten Gespräch und der fachlichen Auseinandersetzung mit dem Thema ergibt. Kapitel 4 überträgt diese theoretischen Konzepte auf die konkrete Implementierung und zeigt anhand der Architektur des eigenen Projekts, wie Choreografie, gezielter LLM-Einsatz und Eskalation an einen Menschen technisch umgesetzt werden können. Kapitel 5 benennt die Grenzen dieses Ansatzes, sowohl auf technischer als auch auf regulatorischer und fachlicher Ebene, bevor Kapitel 6 die Ergebnisse zusammenführt und einordnet.

2 Theorieteil

Dieses Kapitel legt die begrifflichen und konzeptionellen Grundlagen, auf die sich Kapitel 3 und 4 stützen. Es geht dabei nicht um einen vollständigen Überblick über KI in der Versicherungsbranche, sondern gezielt um drei Bausteine, die zur Einordnung des eigenen Projekts nötig sind.

2.1 KI/LLM-Technologien für Prozessautomatisierung

Für die Automatisierung von Geschäftsprozessen mit LLMs sind vor allem drei Aufgabentypen relevant: Klassifikation, strukturierte Extraktion und freie Textgenerierung. Alle drei ersetzen einen Schritt, der bislang eine Person erledigen musste: unstrukturierten Text lesen und daraus eine Entscheidung, einen Wert oder eine Antwort ableiten.

Bei der Klassifikation ordnet das Modell eine Nachricht einer vordefinierten Kategorie zu, meist über eine kurze Anweisung im Prompt, ohne dass dafür ein eigenes Modell trainiert werden muss. Kostina et al. (2025) zeigen in einem Systemvergleich, dass LLMs bei mehrklassigen, semantisch anspruchsvollen Aufgaben klassische Machine-Learning-Verfahren übertreffen. Bei einfachen binären Aufgaben schrumpft dieser Vorteil dagegen deutlich und wird mit spürbar höheren Inferenzzeiten erkauft. Bei sprachlich vielfältigen, aber klar abgrenzbaren Eingangsnachrichten überwiegt dennoch der Vorteil des semantischen Verständnisses gegenüber einem schlankeren, aber starrerem Klassifikator.

Die strukturierte Extraktion geht darüber hinaus. Das Modell liest aus einem Fließtext konkrete Werte heraus und liefert sie in einem vorgegebenen Format zurück, üblicherweise als JSON-Objekt mit benannten Feldern. Möglich wird das durch Modi wie JSON-Mode oder Function-Calling, bei denen die Antwort des Modells auf ein festes Schema eingeschränkt wird. Xu et al. (2024) beschreiben in ihrem Überblick zur generativen Informationsextraktion, dass generative Sprachmodelle klassische, aufgabenspezifisch trainierte Extraktionspipelines zunehmend ersetzen. Sie liefern allein über Prompting brauchbare

strukturierte Ausgaben, ohne für die jeweilige Aufgabe gesondert trainiert zu werden.

Die freie Textgenerierung kommt dort zum Einsatz, wo auch das Ergebnis aus Prosa besteht, etwa eine Zusammenfassung oder eine formulierte Antwort. Eine feste Zielstruktur, an der sich die Ausgabe eindeutig prüfen ließe, fehlt hier. Die Qualität hängt entsprechend stärker vom Prompt-Design ab, und die Ausgabe fällt von Anfrage zu Anfrage unterschiedlicher aus. Das ist ein Umstand, der bei Klassifikation und Extraktion durch das vorgegebene Format deutlich seltener auftritt.

Dass sich Sprachmodelle für genau diese Aufgabentypen produktiv in Versicherungsprozesse einbinden lassen, zeigt auch eine aktuelle Fallstudie aus der Versicherungsbranche. Khayatbashi et al. (2025) dokumentieren den produktiven Einsatz eines LLMs zur automatisierten Identifikation von Schadenbestandteilen in einem realen Schadenprozess. Sie zeigen damit, dass punktuelle LLM-Einsätze an klar abgegrenzten Entscheidungspunkten, statt eines vollständig durchautomatisierten Gesamtprozesses, der in der Praxis verbreitete Ansatz sind.

2.2 Architekturmuster: Orchestrierung vs. Choreografie

Das im Rahmen dieser Arbeit entwickelte System besteht selbst aus mehreren eigenständigen Diensten, weshalb sich die Frage stellt, wie diese Dienste miteinander interagieren. Peltz (2003) unterscheidet dafür bereits im Kontext klassischer Web-Services zwischen zwei grundlegenden Mustern, Orchestrierung und Choreografie. Bei der Orchestrierung steuert eine zentrale Instanz den gesamten Ablauf, ruft die einzelnen Dienste in einer festgelegten Reihenfolge auf und behält dabei den Überblick über den kompletten Prozess. Bei der Choreografie gibt es keine solche zentrale Instanz. Jeder Dienst kennt nur seinen eigenen Ausschnitt des Prozesses, reagiert auf ein eingehendes Ereignis und entscheidet selbst, an wen er das Ergebnis weiterreicht.

Eine Orchestrierung macht den Prozess an einer Stelle sichtbar und leicht nachvollziehbar. Alle Dienste werden an diese zentrale Steuerungslogik gekoppelt. Ändert sich der Ablauf, muss in der Regel nur der Orchestrator angepasst werden, dafür wird er mit wachsender Prozesskomplexität selbst zum kritischen Engpass. Eine Choreografie verteilt diese Entscheidungslogik auf die beteiligten Dienste. Das macht das System insgesamt loser gekoppelt. Erkauft wird dies damit, dass der Gesamtablauf nirgends an einer Stelle dokumentiert oder erzwungen wird, sondern sich erst aus dem Zusammenspiel aller Einzelentscheidungen ergibt. Rudrabhatla (2018) kommt in einem direkten Vergleich beider Ansätze in Microservice-Architekturen zu dem Ergebnis, dass Choreografie insbesondere dann geeignet ist, wenn vergleichsweise wenige Dienste an einem Prozess beteiligt sind. Zudem eignet sich dieser Ansatz gut für Systeme, die fortlaufend um neue Funktionen erweitert werden, ohne dass bestehende Abläufe dafür unterbrochen werden müssen.

In der aktuellen Diskussion um LLM-basierte Multi-Agenten-Systeme taucht dieselbe Unterscheidung unter neuem Namen wieder auf. Tran et al. (2025) beschreiben ein Spektrum von Koordinationsmechanismen zwischen mehreren LLM-Agenten. An einem Ende steht eine zentrale Planungsinstanz, die die Agenten steuert. Am anderen Ende stehen dezentrale Architekturen, in denen jeder Agent auf Basis lokaler Regeln selbstständig handelt. Im Kern ist das wieder das Verhältnis von Orchestrierung zu Choreografie, nur übertragen auf Systeme, in denen Sprachmodelle statt klassischer Web-Services die einzelnen Schritte

ausführen.

2.3 Human-in-the-loop und Automatisierungsgrade

Nicht jeder Schritt eines automatisierten Prozesses muss bzw. kann ohne menschliches Zutun ablaufen. Genau hier setzt der Begriff Human-in-the-loop an. Wu et al. (2021) definieren ihn als einen automatisierten Prozess, der an bestimmten Stellen menschliches Wissen oder menschliche Erfahrung einbezieht, statt vollständig eigenständig zu entscheiden. Human-in-the-loop bedeutet dabei, dass genau festgelegt wird, an welchen Punkten ein Mensch eingreift. Ein Mensch muss also nicht jeden einzelnen Schritt kontrollieren.

Wie viel ein System dabei selbst übernimmt, lässt sich nach dem Modell von Parasuraman, Sheridan und Wickens (2000) differenzieren. Die Autoren unterscheiden vier Funktionsklassen, die automatisiert werden können. Das sind die Informationsbeschaffung, die Informationsanalyse, die Auswahl einer Entscheidung oder Handlung und die Ausführung dieser Handlung. Für jede dieser Klassen lässt sich der Automatisierungsgrad einzeln festlegen, von vollständig manuell bis vollständig autonom. Ein System kann demnach Informationen selbstständig sammeln und analysieren, die eigentliche Entscheidung aber einem Menschen überlassen.

Diese Differenzierung ist relevant, weil Human-in-the-loop in der Praxis nicht immer das hält, was der Begriff verspricht. Tschitschek et al. (2024) zeigen, dass der Mensch in vielen als Human-in-the-loop bezeichneten Systemen keine tatsächliche Entscheidungsmacht besitzt. Er bestätigt in solchen Fällen oft nur eine von der KI bereits vorbereitete Entscheidung. Die Autoren verweisen zudem auf eine Metaanalyse von über hundert Studien, nach der Mensch-KI-Kombinationen bei Entscheidungsaufgaben häufig schlechter abschneiden als der jeweils bessere der beiden Akteure allein. Ob ein Human-in-the-loop-Mechanismus die Prozessqualität tatsächlich verbessert, hängt damit maßgeblich davon ab, an welcher Funktionsklasse und mit wie viel tatsächlichem Entscheidungsspielraum der Mensch eingebunden wird.

3 Beschreibung des Prozesses

Dieses Kapitel beschreibt, wie eine Fallbearbeitung in einer Exklusivagentur ohne jede Automatisierung typischerweise abläuft. Es geht dabei ausschließlich um die beiden in Kapitel 1 eingegrenzten Bereiche, Schadenmeldungen und Serviceanfragen. Auf dieser Grundlage lässt sich anschließend einschätzen, welche Schritte sich für eine Automatisierung eignen und welche nicht.

3.1 Typischer Ablauf einer Fallbearbeitung in Versicherungsagenturen

Die folgende Beschreibung stützt sich auf das in der Einleitung erwähnte Gespräch mit einer in einer Exklusivagentur tätigen Person. Abbildung 1 fasst den im Folgenden beschriebenen Ablauf für beide Bereiche zusammen. Bei einer Schadenmeldung geht die Nachricht zunächst in freier Form beim Sachbearbeiter ein, per Telefon, E-Mail oder Messenger. Er muss aus dieser Nachricht zunächst selbst herauslesen, um welche Art von Schaden es sich handelt,

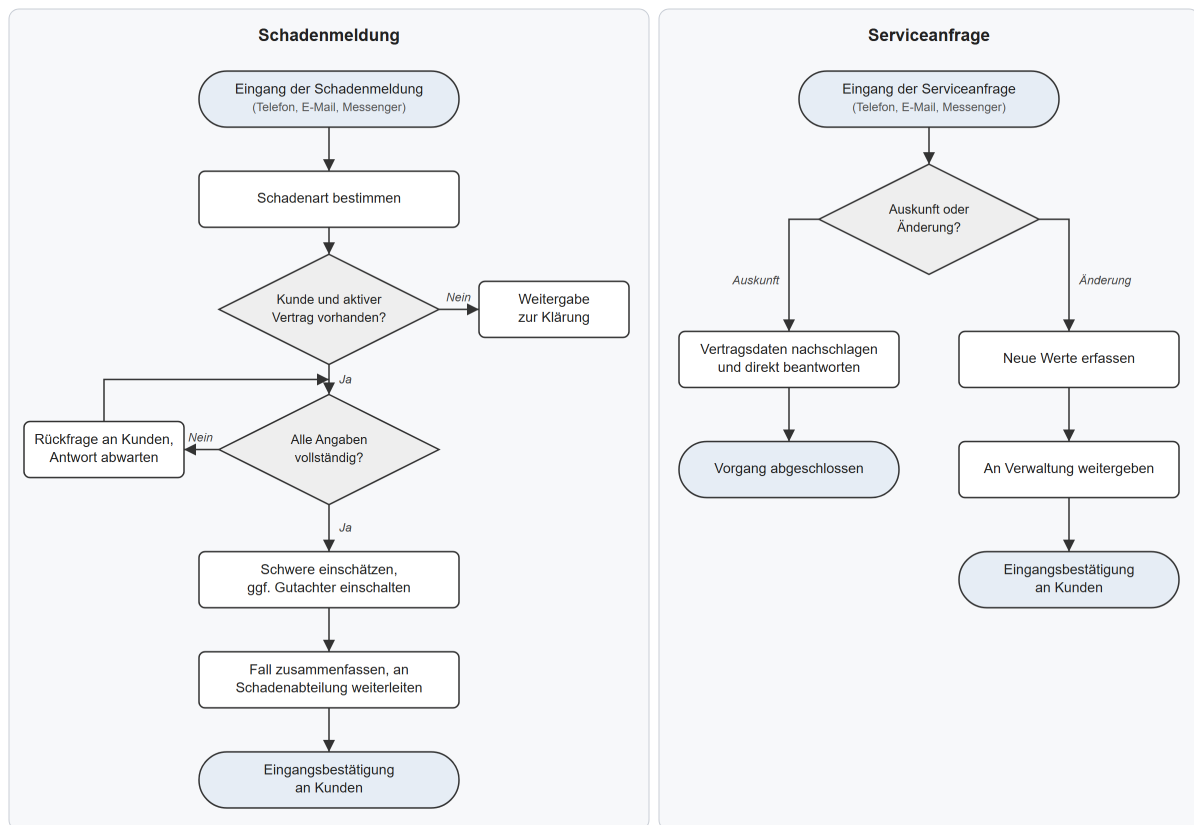


Abbildung 1: Manueller Ablauf von Schadenmeldung und Serviceanfrage in einer Exklusivagentur (eigene Darstellung)

etwa einen Autounfall, einen Wasserschaden oder einen Einbruch. Anschließend prüft er im Bestandssystem, ob die Person überhaupt Kunde ist und ob ein passender, aktiver Vertrag vorliegt. Fehlt eines von beidem, ist die Meldung an dieser Stelle bereits erledigt und wird an die zuständige Stelle zur Klärung weitergegeben. Liegt ein gültiger Vertrag vor, prüft der Sachbearbeiter, ob alle für diese Schadenart notwendigen Angaben vorhanden sind, etwa Datum, Ort und Hergang des Schadens. Fehlen Angaben, muss er selbst eine Rückfrage formulieren und auf die Antwort des Kunden warten, bevor er weiterarbeiten kann. Sind alle Angaben vollständig, schätzt er grob ein, wie schwerwiegend der Schaden ist, und entscheidet, ob zusätzlich ein Gutachter eingeschaltet werden muss. Den Fall fasst er anschließend zusammen und leitet ihn an die zuständige Schadenabteilung weiter. Der Kunde erhält eine Bestätigung, dass seine Meldung eingegangen ist.

Bei einer Serviceanfrage läuft der erste Schritt ähnlich ab. Der Sachbearbeiter liest die Nachricht und muss unterscheiden, ob es sich um eine reine Auskunft handelt, etwa zum Vertragsstatus, oder ob der Kunde tatsächlich etwas ändern möchte, etwa seine Adresse oder sein Fahrzeug. Eine Auskunft kann er meist direkt aus dem Bestandssystem heraus selbst beantworten, womit der Vorgang für ihn abgeschlossen ist. Bei einer Änderung erfasst er die gewünschten neuen Werte und gibt sie an die zuständige Verwaltung weiter, die die Änderung im Bestandssystem vornimmt. Auch hier bestätigt er dem Kunden lediglich den Eingang der Meldung.

3.2 Automatisierbare vs. nicht-automatisierbare Prozessschritte

Betrachtet man den in 3.1 beschriebenen Ablauf durch die in Kapitel 2.1 vorgestellte Brille der drei Aufgabentypen, lässt sich ein Großteil der Schritte einer der drei Kategorien zuordnen. Die Bestimmung der Schadenart und die Unterscheidung zwischen Auskunft und Änderung sind klassische Klassifikationsaufgaben. Die Prüfung, ob alle Pflichtangaben vorliegen, ist strukturierte Extraktion, das Modell muss aus einer Nachricht konkrete Werte herauslesen und mit einer Sollliste abgleichen. Sowohl die Fallzusammenfassung für die Schadenabteilung als auch die Beantwortung einer reinen Auskunft fallen unter freie Textgenerierung.

Nicht jeder automatisierbare Schritt benötigt dabei ein Sprachmodell. Die Prüfung, ob ein Kunde existiert und ob ein aktiver Vertrag vorliegt, ist im Kern ein einfacher Datenbankabgleich. Ein Sprachmodell wäre hier unnötig und würde eine eindeutig regelbasiert lösbare Aufgabe zusätzlich verkomplizieren.

Andere Schritte lassen sich dagegen nicht sinnvoll automatisieren, selbst wenn die vorbereitende Informationsanalyse bereits automatisiert läuft. Die Einschätzung der Schadensschwere (hoch, mittel, niedrig) kann im ersten Schritt das Modell liefern. Die daraus folgende Entscheidung, ob tatsächlich ein Gutachter beauftragt wird, bleibt jedoch eine Ermessensfrage mit finanzieller Tragweite. Dasselbe gilt für die eigentliche Schadenregulierung und für die verbindliche Umsetzung einer Vertragsänderung im Bestandssystem. Nach dem Stufenmodell von Parasuraman et al. (2000) lässt sich das so einordnen, dass Informationsbeschaffung und -analyse automatisiert werden, Entscheidung und Ausführung aber bei diesen Beispielen beim Menschen verbleiben. Der Mensch trägt hier also tatsächlich die inhaltliche Verantwortung für die Entscheidung. Genau das stellen Tschitschek et al. (2024) für viele als Human-in-the-loop bezeichnete Systeme infrage.

Eine Ausnahme bildet die reine Auskunft. Da sie lediglich vorhandene Vertragsdaten wiedergibt und keine Ermessensentscheidung erfordert, lässt sich dieser Prozessschritt von der Informationsbeschaffung bis zur Antwort an den Kunden vollständig automatisieren, ohne dass dabei ein Mensch eingebunden werden muss.

3.3 Veranschaulichung anhand eines Beispielszenarios

Wie sich dieser Ablauf mit einem KI-gestützten System verändert, zeigt das folgende Szenario aus dem eigenen Projekt. Ein fiktiver Kunde, Michael Bauer, schreibt der Agentur per WhatsApp die Nachricht „Hallo, ich hatte gestern einen Unfall mit meinem Auto.“ Diese Nachricht landet nicht zuerst bei einem Sachbearbeiter. Das System erkennt selbstständig, dass es sich um eine Schadenmeldung handelt, genauer um einen Kfz-Unfall. Über die Telefonnummer findet es den Kunden im Bestand und stellt fest, dass eine aktive Vollkaskoversicherung besteht.

Verwertbare Angaben enthält die Nachricht allerdings kaum. Für einen Kfz-Unfall fehlen Datum, Ort und Hergang, das Kennzeichen sowie die Information, ob ein anderes Fahrzeug beteiligt war. Das System schickt dem Kunden deshalb eine Rückfrage, die genau diese fünf Punkte als einfache Fragen enthält. Danach wartet der Fall auf eine Antwort. Bis zu diesem Zeitpunkt war noch kein Mensch beteiligt.

Kurz darauf antwortet der Kunde mit „Das war am 30.08.2026 an der Kreuzung Haupt-

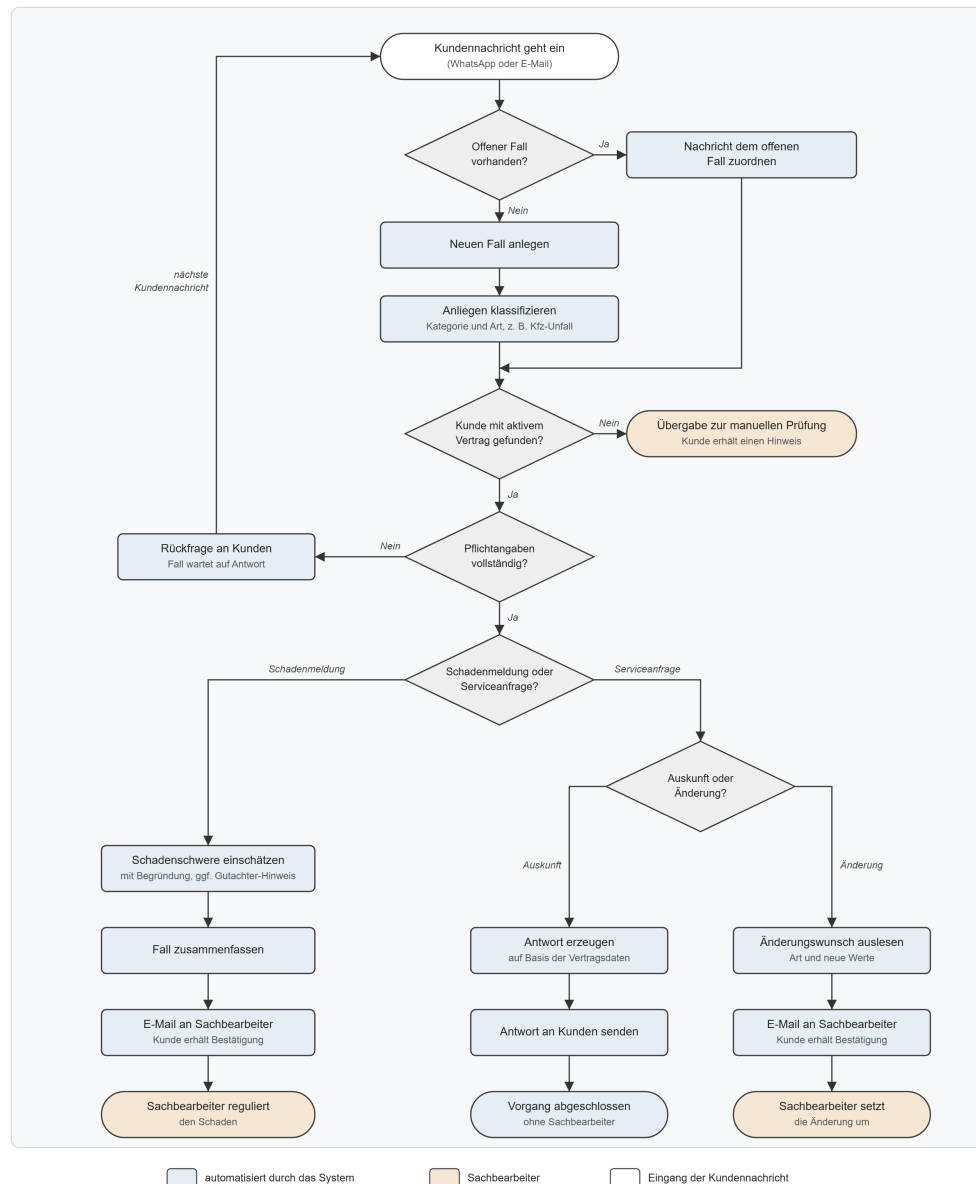


Abbildung 2: KI-gestützter Ablauf von Schadenmeldung und Serviceanfrage im entwickelten System (eigene Darstellung)

str./Bahnhofstr. in Koeln. Ein anderes Fahrzeug ist mir an der roten Ampel hinten aufgefahren, Schaden am Heck, Auto faehrt noch. Mein Kennzeichen ist K-AB 1234.“ Das System ordnet diese Antwort dem noch offenen Fall zu und prüft die Angaben erneut, diesmal anhand beider Nachrichten. Jetzt liegen alle geforderten Informationen vor. Es folgt eine erste Einschätzung der Schadensschwere. Sie fällt mittel aus, weil nur Sachschaden am Heck entstanden ist und das Fahrzeug fahrbereit bleibt. Zusätzlich entsteht eine kurze Zusammenfassung des Falls.

Erst an dieser Stelle kommt ein Mensch ins Spiel. Der Sachbearbeiter erhält eine E-Mail mit allen Falldaten, den Vertragsangaben inklusive Selbstbehalt, der Schwereereinschätzung samt Begründung, der Zusammenfassung und dem vollständigen Nachrichtenverlauf. Parallel bekommt der Kunde eine Bestätigung mit Vorgangsnummer und dem Hinweis, dass sich ein Sachbearbeiter bei ihm meldet. Die eigentliche Regulierung übernimmt der Sachbearbeiter,

ohne die Angaben vorher selbst zusammensuchen zu müssen.

Abbildung 2 zeigt diesen Ablauf in allgemeiner Form, einschließlich der Wege für Serviceanfragen und für Fälle ohne gültigen Vertrag. Das beschriebene Szenario durchläuft darin einmal die Rückfrage-Schleife und folgt anschließend dem Schadenzweig. Im Vergleich zu Abbildung 1 verschiebt sich vor allem der Zeitpunkt, an dem der Sachbearbeiter einsteigt. Im manuellen Ablauf erledigt er jeden Schritt selbst, von der Einordnung der Nachricht bis zur Rückfrage. Im KI-gestützten Ablauf übernimmt er den Fall erst, wenn alle Informationen vollständig und aufbereitet vorliegen.

Daraus ergeben sich mehrere Vorteile gegenüber dem rein manuellen Ablauf. Der Sachbearbeiter wird von Routineschritten entlastet, die wenig fachliches Urteil erfordern, etwa dem Einordnen von Nachrichten, dem Abgleich mit dem Bestand oder dem Nachfragen fehlender Angaben. Seine Zeit fließt stattdessen in die Schritte, für die er tatsächlich gebraucht wird. Der Kunde erhält Rückfragen und Bestätigungen sofort, unabhängig von Öffnungszeiten oder der aktuellen Auslastung der Agentur. Jeder Fall erreicht den Sachbearbeiter zudem in derselben Struktur und mit denselben Pflichtangaben, was Nachfragen innerhalb der Agentur reduziert. Reine Auskünfte beantwortet das System vollständig selbst, sodass sie gar keine Arbeitszeit mehr binden. Schließlich ist der gesamte Dialog mit dem Kunden in der Fallakte dokumentiert und bleibt jederzeit nachvollziehbar. Wie groß die Zeitersparnis im Agenturalltag tatsächlich ausfällt, wurde im Rahmen dieser Arbeit allerdings nicht gemessen.

4 Implementierung und Architektur

Dieses Kapitel zeigt, wie die in Kapitel 2 beschriebenen Konzepte im eigenen Projekt umgesetzt wurden, von den Entwicklungsentscheidungen über die Systemarchitektur bis hin zur konkreten Fallbearbeitung entlang des Szenarios aus Kapitel 3.3.

4.1 Entwicklungsvorgehen und Eigenanteil

Das System wurde mit Claude Code umgesetzt, einem KI-gestützten Entwicklungswerkzeug von Anthropic. Die Arbeitsteilung zwischen Mensch und Werkzeug war dabei klar festgelegt. Bevor die erste Zeile Code entstand, wurde eine ausführliche Spezifikation erarbeitet, die als einzige verbindliche Grundlage der Umsetzung diente. Sie legt die Architekturprinzipien fest, darunter Choreografie statt zentraler Steuerung und die alleinige Datenhoheit eines einzelnen Dienstes. Außerdem beschreibt sie die fünf Dienste mit ihren genauen Abläufen, das Datenmodell eines Falls, die Ordnerstruktur, die Pflichtangaben je Schadenart und die Testkunden für die Demonstration. Ebenso hält sie fest, was bewusst nicht gebaut wird.

Die Architektur ist schrittweise entstanden. Ein erster Entwurf enthielt noch einen eigenen Eingangsdienst, eine zentrale Ablaufsteuerung und automatische Terminvorschläge für Gutachter. Im Laufe der Konzeption wurden diese Elemente gestrichen oder in bestehende Dienste verlagert, weil sie für den Prozess keinen eigenständigen Mehrwert brachten. Jede dieser Entscheidungen ist mit ihrer Begründung in der Spezifikation dokumentiert.

Die Umsetzung selbst erfolgte in sechs Phasen, vom Grundgerüst über die gemeinsamen Werkzeuge und die einzelnen Dienste bis zu den Demonstrationswerkzeugen. Für jede Phase

war vorab ein Prüfkriterium definiert, etwa dass eine eingehende Nachricht eine Fallakte erzeugt. Vor jeder Phase legte Claude Code zunächst einen Plan vor, der geprüft und bei Bedarf korrigiert wurde, bevor Code entstand. Nach jeder Phase wurde das Ergebnis getestet und erst danach in der Versionsverwaltung gesichert.

Der Eigenanteil liegt damit im fachlichen Entwurf, in den Architekturentscheidungen und in der Spezifikation sowie in der Steuerung und Prüfung der Umsetzung. Die Programmierarbeit im engeren Sinne hat überwiegend das Werkzeug übernommen.

4.2 Systemarchitektur: Services und Ordnerstruktur

Das System besteht aus fünf Diensten, die jeweils eine klar abgegrenzte Aufgabe übernehmen (Abbildung 3). Der Case-Service nimmt eingehende Nachrichten entgegen und ist als einziger Dienst von außen erreichbar. Für jeden Kanal gibt es einen eigenen Adapter, der das jeweilige Rohformat von WhatsApp oder E-Mail in ein einheitliches internes Format übersetzt. Ab diesem Punkt kennt der Rest des Systems den Kanal nur noch als Feld. Bevor die Nachricht weiterverarbeitet wird, speichert der Case-Service sie unverändert in einem Posteingangsprotokoll. Anschließend prüft er, ob zum Absender bereits ein offener Fall existiert, und legt andernfalls einen neuen an.

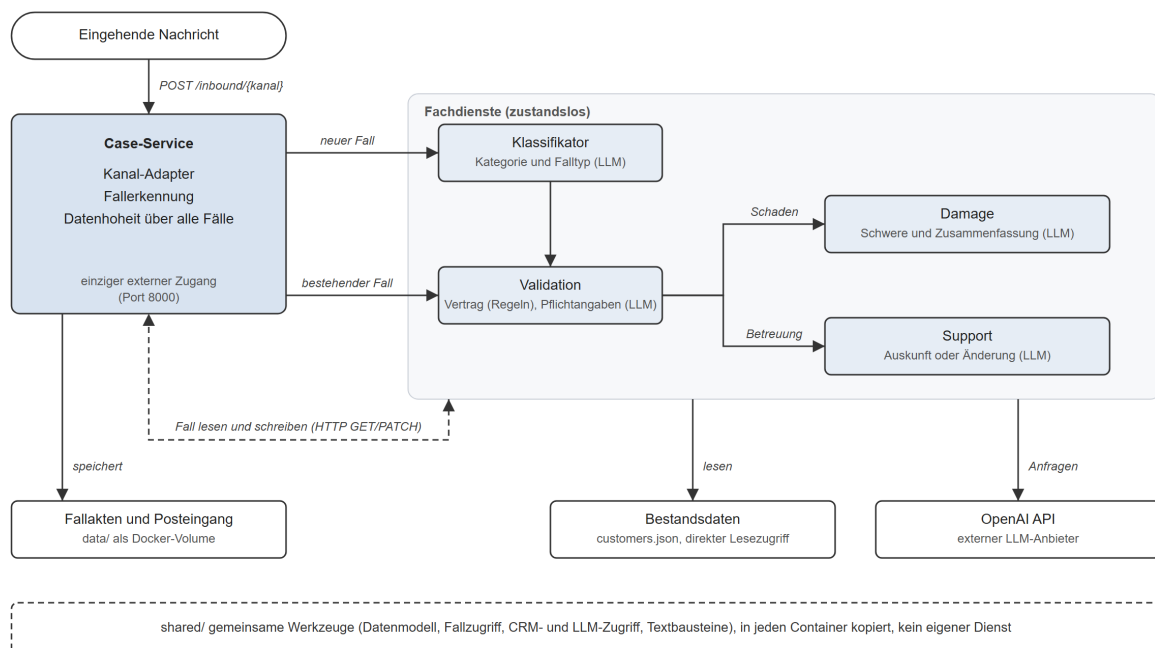


Abbildung 3: Architektur des Systems mit Case-Service, zustandslosen Fachdiensten und externen Ressourcen (eigene Darstellung)

Die vier übrigen Dienste bilden die Fachlogik. Der Klassifikator bestimmt bei neuen Fällen Kategorie und Falltyp. Der Validation-Service prüft Vertrag und Pflichtangaben und fragt bei Bedarf beim Kunden nach. Der Damage-Service schätzt bei Schäden die Schwere ein, erstellt die Zusammenfassung und informiert den Sachbearbeiter. Der Support-Service beantwortet Auskünfte oder leitet Änderungswünsche weiter. Alle vier Dienste sind zustandslos. Sie holen sich den Fall über eine HTTP-Schnittstelle vom Case-Service und schreiben ihre Ergebnisse auf demselben Weg zurück. Die Datenhoheit liegt damit ausschließlich beim Case-Service. Der Fall ist das einzige Gedächtnis des Systems, und alles, was ein späterer Dienst braucht, muss vorher dort eingetragen worden sein. Auf die Bestandsdaten greifen die Fachdienste dagegen direkt zu, da sie dort nur lesen und nichts verändern. Umgesetzt sind alle Dienste in Python mit dem Web-Framework FastAPI, die Sprachmodell-Aufrufe erfolgen über die OpenAI-API.

Die Ordnerstruktur folgt diesem Aufbau. Jeder Dienst hat einen eigenen Ordner, in dem die Datei `main.py` nur den Endpunkt und die Entscheidung über den nächsten Dienst enthält. Die eigentliche Fachlogik liegt in eigenen Dateien daneben, beim Damage-Service etwa getrennt nach Schwereinschätzung, Zusammenfassung und Sachbearbeiter-Mail. Auch die Prompts sind pro Dienst in einer eigenen Datei gebündelt, wobei jeder Prompt genau einer Aufgabe dient. Die Pflichtangaben je Schadenart stehen als reine Konfiguration in einem Feldkatalog. Eine neue Schadenart lässt sich dadurch ergänzen, ohne Programmcode zu ändern. Werkzeuge, die mehr als ein Dienst benötigt, liegen im gemeinsamen Ordner `shared/`. Dazu gehören das Datenmodell, der Zugriff auf Fälle, Bestandsdaten und Sprachmodell sowie die festen Textbausteine für Kundennachrichten. Der Fallzugriff ist dort bewusst nur über HTTP umgesetzt. Damit ist strukturell ausgeschlossen, dass ein Fachdienst die Fallakten direkt verändert. Jeder der fünf Dienste läuft in einem eigenen Docker-Container.

4.3 Choreografie und LLM-Einsatz am Beispielszenario

Der in 3.3 beschriebene Ablauf lässt sich jedem der fünf Dienste konkret zuordnen. Die erste Nachricht trifft beim Case-Service ein. Da zur Telefonnummer noch kein offener Fall existiert, legt er `case_0001` an und ruft allein aufgrund dieser Tatsache den Klassifikator auf, ohne zu wissen, was dieser intern tut. Der Klassifikator führt den ersten LLM-Aufruf aus und bestimmt Kategorie und Falltyp. Er verzweigt danach nicht, sondern ruft in jedem Fall die Validation auf, ein Beispiel dafür, dass ein choreografierter Dienst auch ohne Verzweigung eigenständig entscheidet.

In der Validation läuft zuerst die in 3.2 begründete regelbasierte Vertragsprüfung, danach der zweite LLM-Aufruf, der Pflichtangaben-Check. Da Angaben fehlen, entscheidet die Validation selbst, den Fall zu pausieren, und trägt sich dabei im Fall als wartenden Dienst ein.

Bei der zweiten Nachricht zeigt sich die eigentliche Funktionsweise der Choreografie. Der Case-Service leitet sie nicht nach einer festen Regel weiter, sondern liest aus dem Fall, welcher Dienst wartet, und schickt sie dorthin. Die Validation durchläuft nun denselben Ablauf erneut, diesmal mit vollständigen Angaben, und verzweigt nun nach Kategorie zum Damage-Service.

Dort folgen die letzten beiden LLM-Aufrufe, für die Schwereinschätzung und die Zu-

sammenfassung. Beide Ergebnisse fließen in eine Mail, die der Damage-Service selbst nach einem festen Muster zusammenstellt, ohne weiteren LLM-Aufruf. Über den gesamten Durchlauf hinweg zeigt sich damit eine klare Arbeitsteilung. Vier LLM-Aufrufe entfallen auf Klassifikation, Extraktion, Einschätzung und Textgenerierung, während Vertragsprüfung, Routing und Mailerstellung regelbasiert bleiben.

4.4 Endzustände als Umsetzung von Human-in-the-loop

Jeder Fall endet in einem von drei Zuständen, `assigned`, `manual_review` oder `closed`. Diese setzen die in Kapitel 2.3 diskutierte Frage, wie viel Entscheidungsspielraum beim Menschen verbleibt, konkret im Code um.

`Closed` steht nur bei einer reinen Auskunft, gesetzt vom Support-Service. Das System hat dann alle vier Funktionsklassen nach Parasuraman et al. (2000) selbst übernommen, von der Informationsbeschaffung über die Analyse der Vertragsdaten bis zur formulierten Antwort an den Kunden. Kein Sachbearbeiter wird einbezogen.

`Assigned` steht bei Schaden und Änderungsmeldung, je nachdem gesetzt vom Damage- oder vom Support-Service. Beide Dienste haben zu diesem Zeitpunkt bereits Informationen gesammelt und ausgewertet, für den Schaden als Severity-Einschätzung und Zusammenfassung, für die Änderungsmeldung als extrahierte Werte. Die eigentliche Entscheidung und ihre Ausführung, die Schadenregulierung oder die verbindliche Änderung im Bestandssystem, bleiben aber vollständig aus. Tschitschek et al. (2024) kritisieren, dass der Mensch in vielen Human-in-the-loop-Systemen nur eine vom Modell bereits getroffene Entscheidung bestätigt. Hier liegt der Fall anders, das System schlägt keine Entscheidung vor, die nur noch abgenickt werden müsste, es liefert ausschließlich die Grundlage dafür. Die Entscheidung selbst entsteht erst beim Sachbearbeiter.

`Manual_review` hat einen anderen Charakter als die beiden anderen Zustände. Der Validation-Service setzt ihn, wenn zu einem Absender kein Kunde oder kein aktiver Vertrag im Bestand gefunden wird. Die eigentliche Prüfung kann dann gar nicht erst beginnen, der Fall geht sofort an den Sachbearbeiter. Anders als bei `assigned` ist das keine gestaltete Grenze zwischen automatisierbaren und nicht automatisierbaren Funktionsklassen, eher eine Ausnahmebehandlung für einen Fall, der sich gar nicht regulär durch den Prozess bewegen kann.

5 Grenzen und Ausblick

Dieses Kapitel schließt die Betrachtung des Systems ab. Zunächst werden die wichtigsten Grenzen benannt, an die die aktuelle Umsetzung stößt, technische wie regulatorische. Der anschließende Ausblick zeigt, in welche Richtung sich das System von hier aus sinnvoll weiterentwickeln ließe.

5.1 Grenzen

Es gibt keine echte Anbindung an WhatsApp oder ein E-Mail-Postfach, kein echtes Bestandssystem und keine Fehlerbehandlung. Nachrichten, Kunden und Fälle sind für diese Arbeit nachgebildet, nicht mit realen Systemen verbunden. Ein Aufruf, der fehlschlägt, bricht sichtbar ab, statt es erneut zu versuchen.

Datenschutzrechtlich gehen Kundendaten für jeden LLM-Aufruf an einen externen Anbieter außerhalb der EU. Das wirft die Frage nach einer Auftragsverarbeitung und einem zulässigen Drittlandtransfer auf, die diese Arbeit nicht abschließend beantwortet.

Fachlich besteht schließlich die Gefahr, dass der Sachbearbeiter eine vom LLM erstellte Einschätzung oder Zusammenfassung ungeprüft übernimmt. Parasuraman und Manzey (2010) beschreiben dieses Verhalten als Automation Bias und zeigen, dass es weder durch Erfahrung noch durch Training zuverlässig verhindert werden kann. Gerade weil das System in Kapitel 2.3 als Beispiel für echte menschliche Entscheidungsgewalt diente, ist dieser Effekt hier besonders relevant. Diese Entscheidungsgewalt nützt wenig, wenn die zugrunde liegende Einschätzung unhinterfragt übernommen wird.

5.2 Ausblick

Naheliegend wäre zunächst ein Retrieval-Augmented-Generation-Ansatz für die Auskunft-Funktion. Statt nur die wenigen strukturierten Vertragsfelder heranzuziehen, könnte das System dann auf die tatsächlichen Versicherungsbedingungen zugreifen und differenziertere Fragen beantworten.

An der Klassifikationsstelle wiederum stellt sich die Frage, ob überhaupt ein allgemeines Sprachmodell nötig ist. Der Klassifikator entscheidet nur zwischen zwei Kategorien und einer festen Menge an Falltypen aus dem Feldkatalog, ohne dabei Fließtext zu erzeugen. Das Anfang September 2026 vorgestellte Modell „Jev“ von TypeSafe AI ist genau für solche Entscheidungen gebaut. Es liefert laut Hersteller typisierte Werte aus einem vorab festgelegten Schema statt freien Textes und kann dadurch keinen Wert außerhalb dieses Schemas ausgeben. An der Klassifikationsstelle eingesetzt, wäre damit strukturell ausgeschlossen, dass ein Falltyp entsteht, der nicht im Feldkatalog steht. Da es sich bei Jev um ein sehr junges, bislang nur vom Hersteller selbst beschriebenes Produkt ohne unabhängige Prüfung handelt, ist diese Einschätzung mit Vorsicht zu lesen. Für die übrigen LLM-Aufrufe im System eignet sich ein solches Modell dagegen nicht. Der Pflichtangaben-Check extrahiert Werte wie Datum oder Adresse aus freiem Text, es gibt keine feste Antwortmenge, aus der ausgewählt werden könnte. Zusammenfassung und Auskunft erzeugen zudem Fließtext für Menschen. Dort bleibt ein allgemeines Sprachmodell notwendig.

Neue Dienste lassen sich vergleichsweise einfach ergänzen, weil jeder Fachdienst zustandslos ist und nur über den gemeinsamen **CaseClient** mit dem Case-Service spricht. Ein zusätzlicher Dienst muss lediglich in die Weiterleitungsentscheidung eines bestehenden Dienstes eingetragen werden, ohne dass andere Dienste angepasst werden müssten. Dieselbe Zustandslosigkeit erlaubt es außerdem, mehrere Instanzen eines Fachdienstes parallel zu betreiben, wenn das Nachrichtenaufkommen wächst. Nur der Case-Service hält Zustand und bliebe bei einer solchen Skalierung der einzige Engpass.

Mit einer wachsenden Zahl an Diensten und Kanälen stellt sich schließlich erneut die

Frage aus Kapitel 2.2. Rudrabhatla (2018) begründet den Vorteil der Choreografie gerade mit einer überschaubaren Zahl beteiligter Dienste. Bei deutlich mehr Diensten könnte ein Orchestrator oder ein hybrides Modell, in dem einzelne Teilbereiche orchestriert und nur die Bereiche dazwischen choreografiert ablaufen, die bessere Wahl sein.

6 Fazit

Die Arbeit ist aus der Frage entstanden, wie sich KI in den Exklusivagenturen eines Versicherers sinnvoll einsetzen lässt. Das Ergebnis ist ein lauffähiges System aus fünf eigenständigen Diensten, die Schadenmeldungen und Serviceanfragen automatisiert vorverarbeiten und aufbereitet an einen Sachbearbeiter übergeben. Ein Sprachmodell kommt dabei gezielt an vier Stellen zum Einsatz, für Klassifikation, Pflichtangaben-Check, Schwereereinschätzung und Textgenerierung, während Vertragsprüfung und Weiterleitung regelbasiert bleiben. Am Ende jedes Falls steht einer von drei Zuständen, eine automatisch geschlossene Auskunft, ein an den Sachbearbeiter übergebener Fall bei Schaden oder Änderungsmeldung oder eine manuelle Prüfung ohne gültigen Vertrag. Umgesetzt wurde das System containerisiert mit Python, FastAPI und der OpenAI-API, mithilfe von Claude Code auf Basis einer eigens erarbeiteten Spezifikation. Wie zuverlässig und wirtschaftlich sich das in einer echten Agentur auswirken würde, kann diese Arbeit nicht beantworten, dafür fehlt die Evaluation unter realen Bedingungen. Gezeigt hat sie aber, dass sich der manuelle Ablauf technisch so zerlegen lässt, dass ein Sachbearbeiter nur noch dort gebraucht wird, wo es tatsächlich auf sein Urteil ankommt.

Quellenverzeichnis

- Khayatbashi, S., Sjölin, V., Granåker, A. & Jalali, A. (2025) *AI-Enhanced Business Process Automation: A Case Study in the Insurance Domain Using Object-Centric Process Mining*. In: Proceedings of BPMDS 2025. Springer. arXiv:2504.17295. Verfügbar unter: <https://arxiv.org/abs/2504.17295>
- Kostina, A., Dikaiakos, M. D., Stefanidis, D. & Pallis, G. (2025) *Large Language Models for Text Classification: Case Study and Comprehensive Review*. arXiv:2501.08457. Verfügbar unter: <https://arxiv.org/abs/2501.08457>
- Parasuraman, R. & Manzey, D. H. (2010) Complacency and Bias in Human Use of Automation: An Attentional Integration. *Human Factors*, 52(3), S. 381–410. DOI: <https://doi.org/10.1177/0018720810376055>
- Parasuraman, R., Sheridan, T. B. & Wickens, C. D. (2000) A Model for Types and Levels of Human Interaction with Automation. *IEEE Transactions on Systems, Man, and Cybernetics – Part A: Systems and Humans*, 30(3), S. 286–297.
- Peltz, C. (2003) Web Services Orchestration and Choreography. *Computer*, 36(10), S. 46–52. DOI: <https://doi.org/10.1109/MC.2003.1236471>
- Rudrabhatla, C. K. (2018) Comparison of Event Choreography and Orchestration Techniques in Microservice Architecture. *International Journal of Advanced Computer Science and Applications*, 9(8). Verfügbar unter: <https://thesai.org/Publications/ViewPaper?Volume=9&Issue=8&Code=IJACSA&SerialNo=4>
- Tran, K.-T., Dao, D., Nguyen, M.-D., Pham, Q.-V., O’Sullivan, B. & Nguyen, H. D. (2025) *Multi-Agent Collaboration Mechanisms: A Survey of LLMs*. arXiv:2501.06322. Verfügbar unter: <https://arxiv.org/abs/2501.06322>
- Tschiatschek, S., Stamboliev, E., Schmude, T., Coeckelbergh, M. & Koesten, L. (2024) *Challenging the Human-in-the-loop in Algorithmic Decision-making*. arXiv:2405.10706. Verfügbar unter: <https://arxiv.org/abs/2405.10706>
- Wu, X., Xiao, L., Sun, Y., Zhang, J., Ma, T. & He, L. (2021) *A Survey of Human-in-the-loop for Machine Learning*. arXiv:2108.00941. Verfügbar unter: <https://arxiv.org/abs/2108.00941>
- Xu, D., Chen, W., Peng, W., Zhang, C., Xu, T., Zhao, X., Wu, X., Zheng, Y., Wang, Y. & Chen, E. (2024) Large Language Models for Generative Information Extraction: A Survey. *Frontiers of Computer Science*. arXiv:2312.17617. Verfügbar unter: <https://arxiv.org/abs/2312.17617>

Erklärung

Hiermit erkläre ich, dass ich die vorliegende Arbeit eigenständig verfasst und keine anderen als die angegebenen Quellen verwendet habe. Alle Teile der Arbeit, die wörtlich oder sinngemäß aus den Werken anderer Autorinnen und Autoren übernommen wurden, sind als solche gekennzeichnet. Diese Arbeit wurde weder ganz noch teilweise einer anderen Prüfungsbehörde vorgelegt und bisher nicht veröffentlicht.

27. September 2026

Okan Ayhan (30433120)